

WEEK 02 MODULE

Programming Practice

3. Sort Employee Salaries (Bubble Sort)

que. 2 What is Sorting?

- Sorting is the technique of arranging data elements in a particular order according to a chosen criterion, such as numerical value, alphabetical order, or date.

Types of Sorting :-

- i. Bubble Sort
- ii. Selection Sort
- iii. Insertion Sort
- iv. Merge Sort
- v. Quick Sort
- vi. Heap Sort

Common

1. Bubble Sort

Definition : Bubble Sort is a simple sorting algorithm that repeatedly compares adjacent elements and swaps them if they are in the wrong order. After each pass, the largest unsorted element "bubbles up" to its correct position.

Key Points:-

- Compares **adjacent elements**.
- Swaps elements when they are in the wrong order.
- After each pass, the largest element reaches its correct position.
- Easy to understand and implement.
- Not efficient for large datasets.

How It Works:-

Consider the array: 5 3 8 1

Pass 1

Compare 5 and 3

$5 > 3 \rightarrow \text{Swap}$
3 5 8 1

Compare 5 and 8

$5 < 8 \rightarrow \text{No Swap}$
3 5 8 1

Compare 8 and 1

$8 > 1 \rightarrow \text{Swap}$
3 5 1 8
Largest element (8) is now in its

Pass 2

Compare 3 and 5

$3 < 5 \rightarrow \text{No Swap}$

Compare 5 and 1

$5 > 1 \rightarrow \text{Swap}$

3 1 5 8

Pass 3

Compare 3 and 1

$3 > 1 \rightarrow \text{Swap}$

1 3 5 8

Array is sorted.

1 3 5 8

Problem Statement

A company wants to sort employee salaries in ascending order for payroll analysis.

1. Problem Analysis

A company maintains employee salary records for payroll analysis and administrative purposes. However, the salaries may not be stored in any particular order. To simplify analysis, comparison, and reporting, the salary records need to be arranged in ascending order.

The challenge is to reorganize the salaries from the smallest value to the largest value while ensuring that every salary is placed in its correct position.

2. Algorithm Selection Analysis

Bubble Sort is selected to arrange the employee salaries in ascending order.

The algorithm repeatedly compares adjacent salary values and swaps them whenever they are found in the wrong order. Through multiple passes, larger salary values gradually move toward the end of the array.

Reasons for selecting Bubble Sort:

- Simple and easy to understand.
- Suitable for demonstrating fundamental sorting concepts.
- Performs sorting through repeated comparisons and swaps.
- Does not require additional data structures.

3. Core Idea / Logic

Bubble Sort works by repeatedly comparing adjacent elements in the array.

- If the current element is greater than the next element, the two elements are swapped.
- After one complete pass, the largest element reaches its correct position at the end of the array.
- The process repeats for the remaining unsorted portion.
- Sorting is completed when all elements are arranged in ascending order.

The larger values gradually "bubble" toward the end of the array, which gives the algorithm its name.

4. Execution Flow

Input

- Read the number of employee salary records (N).
- Store all salary values in an array.

Processing

- Traverse the array multiple times.
- Compare adjacent salary values.
- If the current salary is greater than the next salary, swap them.
- After each pass, the largest unsorted salary moves to its correct position.
- Repeat the process until all salaries are arranged in ascending order.

Output

- Display the sorted salary records in ascending order.

Step-by-Step Program Execution

Using the sample input:

5

45000 32000 55000 28000 60000

Step 1

Read the number of employee salary records.

User enters 5.

```
int n = sc.nextInt();
```

n=5

Step 2

Create an array of size n.

```
int arr[] = new int[n];
```

Step 3

Accept all salary values from the user and store them in the array.

45000 32000 55000 28000 60000

Array becomes:

Index : 0 1 2 3 4

Value : 45000 32000 55000 28000 60000

Step 4

Start the outer loop to control the number of passes required for sorting.

Step 5

Start the inner loop to compare adjacent salary values.

Step 6

Compare the current salary with the next salary.

Step 7

If the current salary is greater than the next salary , swap both values.

Step 8

Continue comparing adjacent elements until the current pass is completed.

Step 9

After each pass, the largest unsorted salary reaches its correct position at the end of the array.

Step 10

Repeat the process for the remaining unsorted portion of the array.

Step 11

After all passes are completed, the salaries become fully sorted in ascending order.

Step 12

Display the sorted salary records.

5. Key Observations

- The array does not need to be initially sorted.
- Bubble Sort compares adjacent elements only.
- Swapping occurs whenever two neighboring elements are found in the wrong order.

- After the first pass, the largest element is guaranteed to be placed at the last position.

- After the second pass, the second-largest element reaches its correct position.
- With each pass, the unsorted portion of the array becomes smaller.
- Best-case scenario occurs when the array is already sorted and no swaps are required.
- Worst-case scenario occurs when the array is arranged in reverse order, requiring the maximum number of comparisons and swaps.
- Bubble Sort is simple to implement but becomes inefficient for large datasets due to repeated comparisons.

6. Program Structure Analysis

i. Variable n

```
int n;
```

Purpose:

- Stores the total number of employee salary records.
- Determines the size of the array.
- Controls the number of inputs and sorting operations.

ii. Array arr[]

```
int arr[] = new int[n];
```

Purpose:

- Stores all employee salary values.
- Provides indexed access to each salary during sorting.
- Holds both unsorted and sorted salary records.

iii. Input Loop

```
for(int i = 0; i < n; i++)
{
    arr[i] = sc.nextInt();
}
```

Purpose:

- Accepts salary values from the user.
- Stores each salary in the array.
- Ensures all records are available before sorting begins.

iv. Outer Loop

```
for(int i = 0; i < n - 1; i++)
```

Purpose:

- Controls the number of sorting passes.
- Ensures that every element gets an opportunity to reach its correct position.
- Repeats the comparison process until the array becomes sorted.

v. Inner Loop

```
for(int j = 0; j < n - i - 1; j++)
```

Purpose:

- Performs comparisons between adjacent elements.
- Traverses only the unsorted portion of the array.
- Reduces unnecessary comparisons after each pass.

vi. Comparison Condition

```
if(arr[j] > arr[j + 1])
```

Purpose:

- Checks whether two adjacent salary values are in the wrong order.
- Determines whether swapping is required.

vii. Temporary Variable

```
int temp;
```

Purpose:

- Temporarily stores one value during swapping.
- Prevents data loss while exchanging elements.

viii. Swapping Process

```
temp = arr[j];  
arr[j] = arr[j + 1];  
arr[j + 1] = temp;
```

Purpose:

- Exchanges two adjacent salary values.
- Moves larger values toward the end of the array.
- Gradually arranges salaries in ascending order.

ix. Output Loop

```
for(int i = 0; i < n; i++)
```

Purpose:

- Traverses the sorted array.
- Displays salary records in ascending order.

Why $n - i - 1$?

This is the most important Bubble Sort observation.

```
for(int j = 0; j < n - i - 1; j++)
```

After each pass:

- The largest unsorted element reaches its correct position.
- That position no longer needs comparison.

Example:

After Pass 1:

12 28 32 45 55
 ↑
 Fixed Position

After Pass 2:

12 28 32 45 55
 ↑ ↑
 Fixed Positions

Therefore, the inner loop becomes shorter after every pass, which avoids unnecessary comparisons and improves efficiency.

7. Program

```
package week2_module;
import java.util.Scanner;
public class EmployeeSalariesBubbleSort {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        // Input number of employee salaries
        System.out.print("Enter the number of employee salaries: ");
        int n = sc.nextInt();

        // Create array
```

```

int arr[] = new int[n];

// Input salary records
System.out.print("Enter the salary records of " + n + " employees: ");
for(int i = 0; i < n; i++){
    arr[i] = sc.nextInt();
}

// Bubble Sort
for(int i = 0; i < n - 1; i++) //outer loop
{
    for(int j = 0; j < n - i - 1; j++) //inner loop
    {
        if(arr[j] > arr[j + 1]) //swap
        {
            int temp = arr[j];
            arr[j] = arr[j + 1];
            arr[j + 1] = temp;
        }
    }
}

// Display sorted salaries
System.out.println("Sorted Employee Salaries:");

for(int i = 0; i < n; i++){
    System.out.print(arr[i] + " ");
}

sc.close();
}
}

```

8. Program's output

```

Enter the number of employee salaries: 5
Enter the salary records of 5 employees: 45000
32000
55000
28000
60000
Sorted Employee Salaries:
28000 32000 45000 55000 60000

```

9. Screenshot

- program

```
EmployeeSalariesBubbleSort.java X
1 package week2_module;
2 import java.util.Scanner;
3 public class EmployeeSalariesBubbleSort {
4
5     public static void main(String[] args) {
6         Scanner sc = new Scanner(System.in);
7         System.out.print("Enter the number of employee salaries: "); // Input number of employee salaries
8         int n = sc.nextInt();
9         int arr[] = new int[n]; // Create array
10        System.out.print("Enter the salary records of " + n + " employees: "); // Input salary records
11        for(int i = 0; i < n; i++)
12        {
13            arr[i] = sc.nextInt();
14        }
15
16        // Bubble Sort
17        for(int i = 0; i < n - 1; i++) //outer loop
18        {
19            for(int j = 0; j < n - i - 1; j++) //inner loop
20            {
21                if(arr[j] > arr[j + 1]) //swap
22                {
23                    int temp = arr[j];
24                    arr[j] = arr[j + 1];
25                    arr[j + 1] = temp;
26                }
27            }
28        }
29
30        // Display sorted salaries
31        System.out.println("Sorted Employee Salaries:");
32
33        for(int i = 0; i < n; i++)
34        {
35            System.out.print(arr[i] + " ");
36        }
37
38        sc.close();
39    }
40 }
```

- program's output

```
Problems Javadoc Declaration Console X
<terminated> EmployeeSalariesBubbleSort [Java Application] /Library/Ja
Enter the number of employee salaries: 5
Enter the salary records of 5 employees: 45000
32000
55000
28000
60000
Sorted Employee Salaries:
28000 32000 45000 55000 60000
```